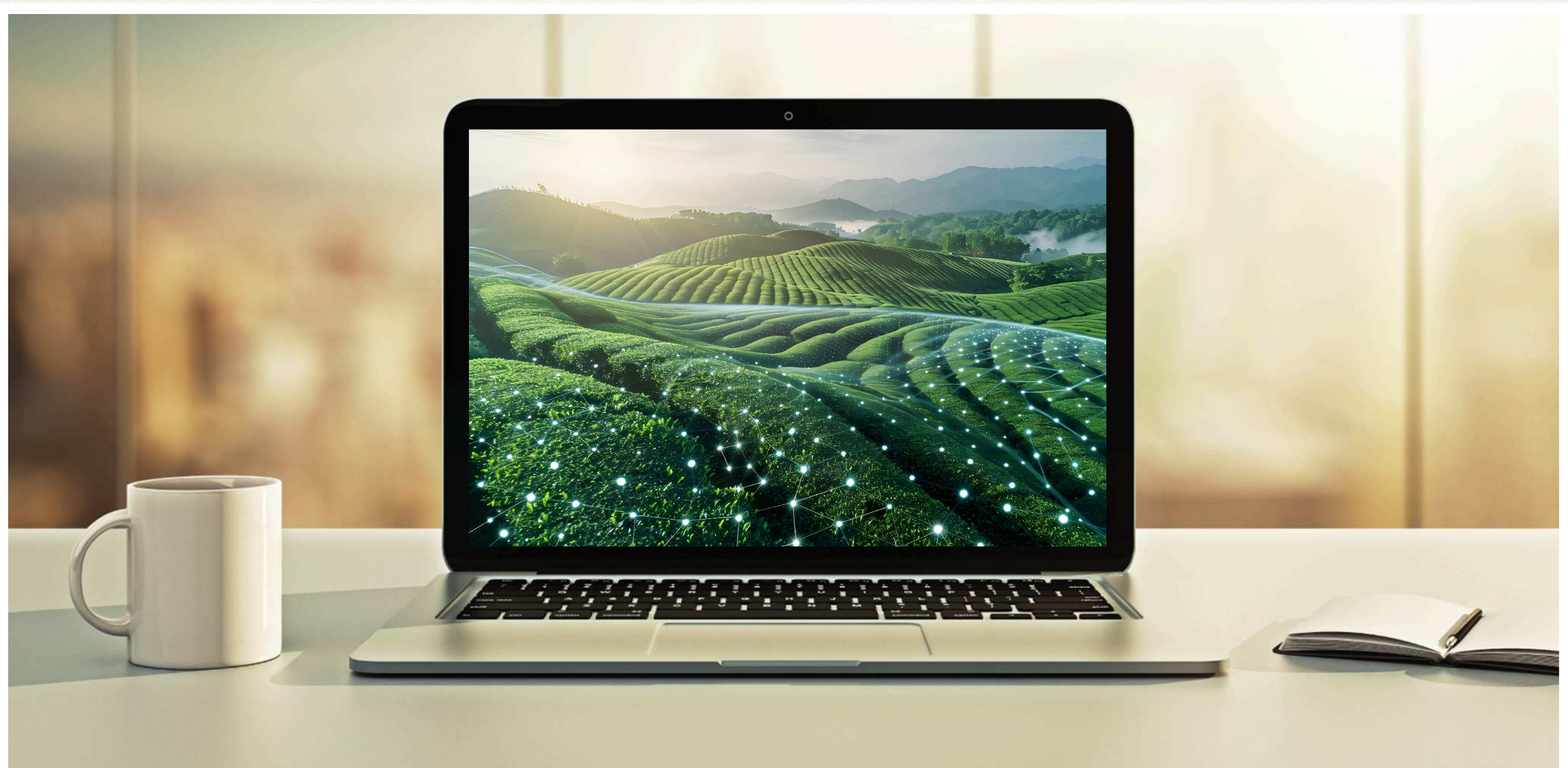


Books Project 2



Project 2

- Project two will still be focused on creating Book API Endpoints
- Continued Education includes:
 - GET, POST, PUT, DELETE Request Methods
- New Information will include:
 - Data Validation, Exception Handling, Status Codes, Swagger Configuration, Java Request Objects

Creating a new books project

- We will update our class called Book
- We will be using these books throughout this project:

```
public class Book {  
    private long id;  
    private String title;  
    private String author;  
    private String category;  
    private int rating;  
  
    public Book(long id, String title, String author, String category, <...> ) {  
        this.id = id;  
        this.title = title;  
        this.author = author;  
        this.category = category;  
        <...>  
    }  
}
```

< Getters and Setters >

HTTP Request Methods

CRUD

HTTP Request Methods

Create



POST

Read



GET

Update



PUT

Delete



DELETE

Enhance Book Objects



Enhancing our Book object

- We will enhance our class called Book
- We will be using these books throughout this project:

```
public class Book {  
    private long id;  
    private String title;  
    private String author;  
    private String category;  
    private int rating;  
}
```

Unique identifier

New fields

```
public Book(long id, String title, String author, String category, <...> ) {  
    this.id = id;  
    this.title = title;  
    this.author = author;  
    this.category = category;  
    <...>  
}
```

< Getters and Setters >

New fields

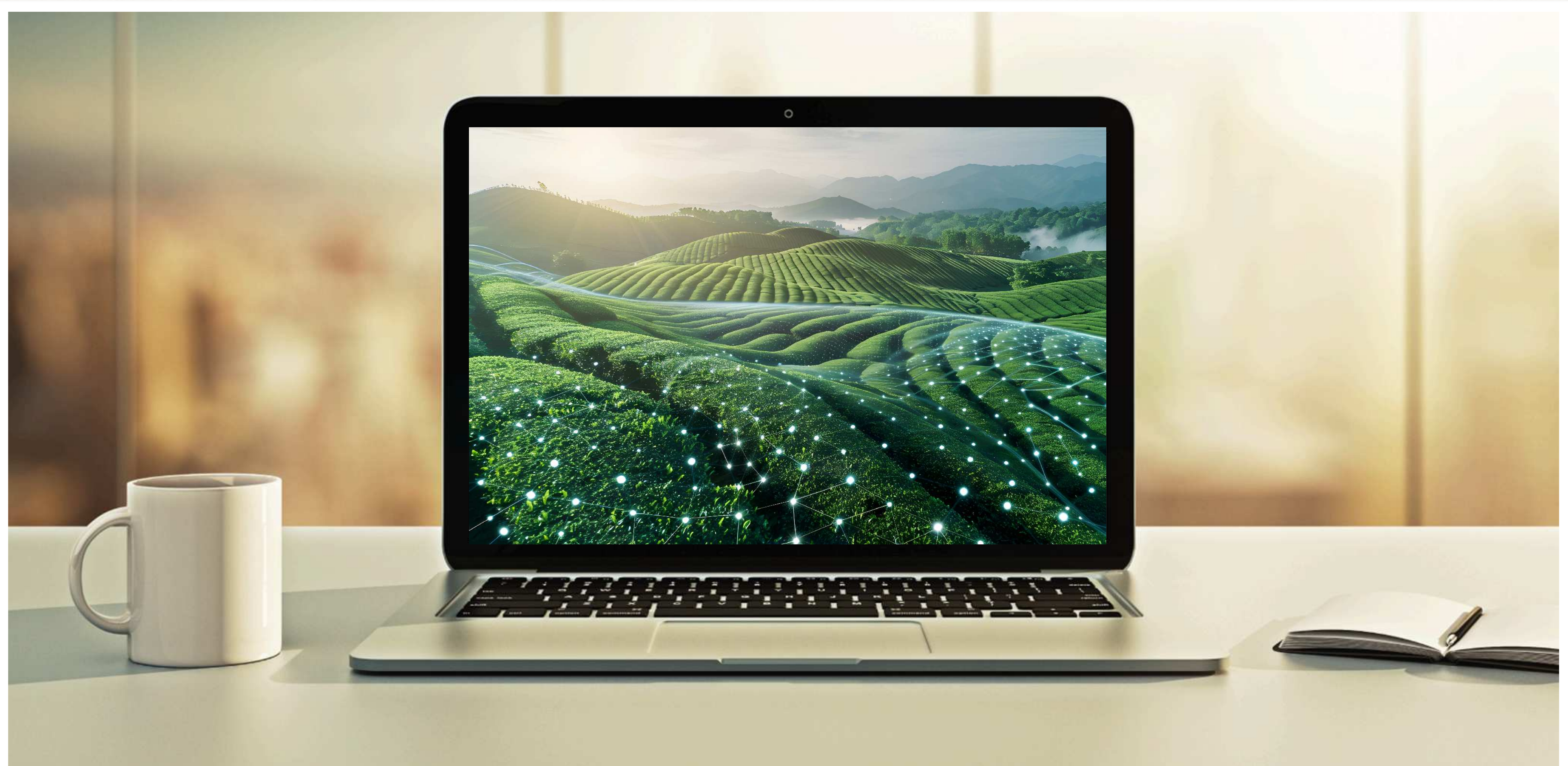
- Creating and enhancing books to learn the basics of Spring Boot Rest

```
public void initializeBooks() {  
    books.addAll(List.of(  
        new Book("Title One", "Author One", "science"),  
        new Book("Title Two", "Author Two", "science"),  
        new Book("Title Three", "Author Three", "history"),  
        new Book("Title Four", "Author Four", "math"),  
        new Book("Title Five", "Author Five", "math"),  
        new Book("Title Six", "Author Two", "math")  
    ));  
}
```

```
public void initializeBooks() {  
    books.addAll(List.of(  
        add(new Book(1, "Java Master", "Eric R.", "Comp Science", 5));  
    ));  
}
```

New fields

API Endpoints ID



Path Variables

Read



Get

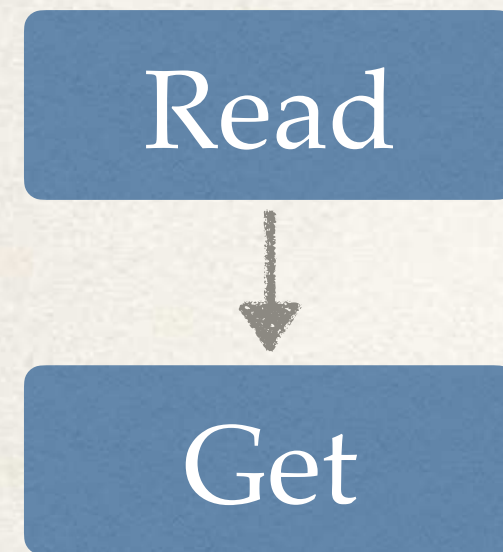
REQUEST:

URL : localhost:8080 / api / books / title%20four



```
@GetMapping("/{title}")
public Book getBookByBook7 (@PathVariable String title) {
    return book.stream()
        .filter(book -> book.get().equalsIgnoreCase(title))
        .findFirst()
        .orElse(null);
}
```


Path Parameters



REQUEST:

URL : localhost:8080 / api / books / 2

URL : localhost:8080 / api / books / title%20four

- Endpoints match.
- We need to change to not “confuse” Spring Boot.

URL : localhost:8080 / api / books / {dynamic}

URL : localhost:8080 / api / books / 2

URL : localhost:8080 / api / books / title%20four

Path Variables

Read



Get

REQUEST:

URL : localhost:8080 / api / books / 2

```
@GetMapping("/{id}")
public Book getBookById(@PathVariable long id) {
    return book.stream()
        .filter(book -> book.getId() == id)
        .findFirst()
        .orElse(null);
}
```

RESPONSE:

```
{
  "id": 2,
  "title": "Title Two",
  "author": "Author Two",
  "category": "science",
  "rating": 3
}
```


Path Variables

REQUEST:

URL : localhost:8080 / api / books / 2

Update

Put

```
@PutMapping("/{id}")
public void updateBook(@PathVariable long id, @RequestBody Book updatedBook) {
    for (int i = 0; i < books.size(); i++) {
        if (book.get(i).getId() == id) {
            books.set(i, updatedBook);
            return;
        }
    }
}
```


Path Variables

REQUEST:

URL : localhost:8080 / api / **books / 2**

Delete

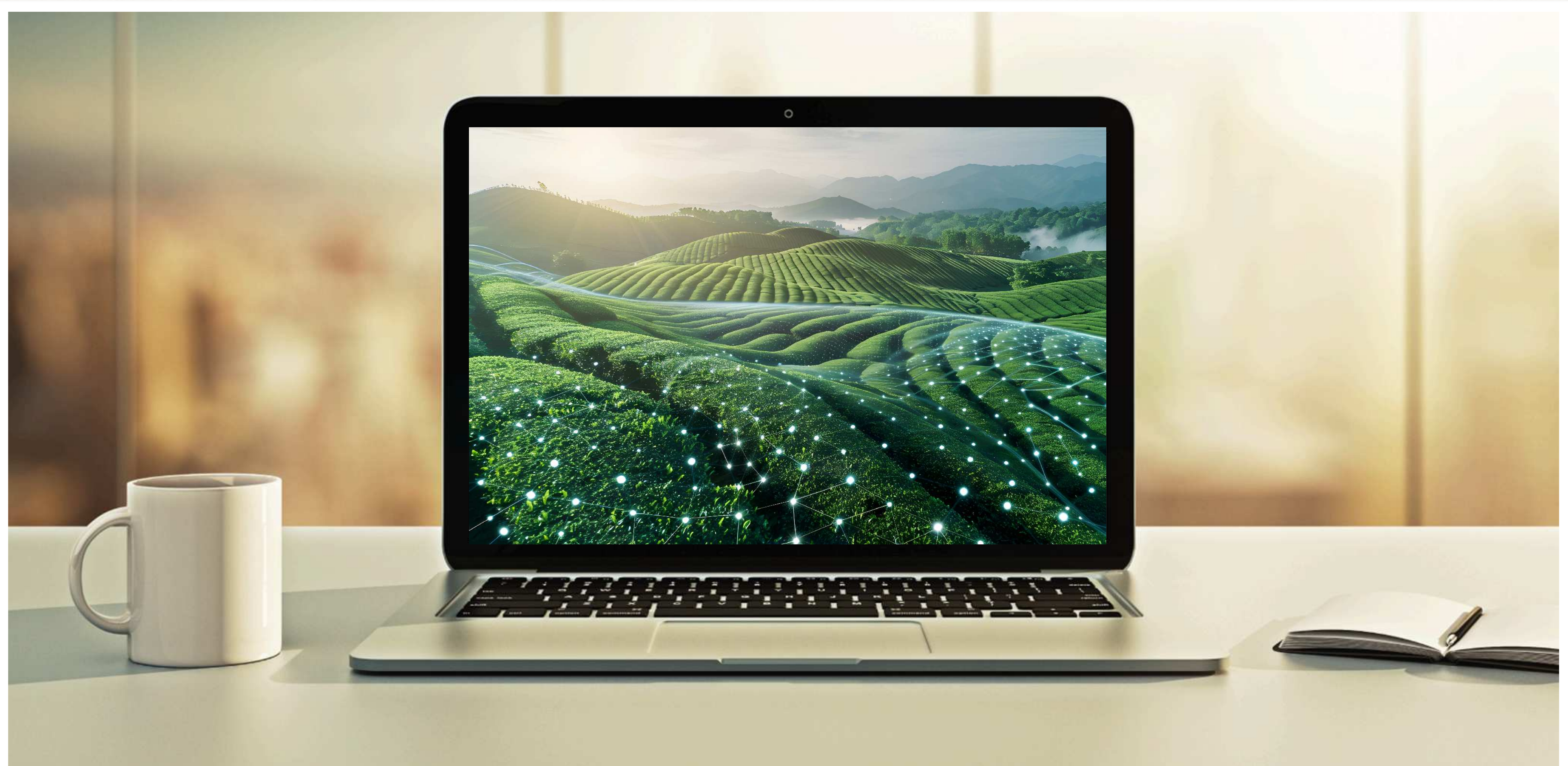


Delete



```
@DeleteMapping("/{id}")  
public void deleteBook(@PathVariable long id) {  
    books.removeIf(book -> book.getId() == id);  
}
```


BookRequest Object



What are Request Objects

- Request objects are designed specifically to capture and validate incoming requests from HTTP.
- Only contain data necessary for the request.
- Only used for sending data to server, unlike entity objects which typically have more information and has a tighter connection to business logic.
- Improves security.
- Flexible by decoupling entity models

Book Entity

```
public class Book {  
    private long id;  
    private String title;  
    private String author;  
    private String category;  
    private int rating;  
  
    public Book(String title, String author, String category, <...> ) {  
        this.title = title;  
        this.author = author;  
        this.category = description;  
        <...>  
    }  
}
```

< Getters and Setters >

- Includes an id ..
- Id is not needed for POST Requests.

Book Entity

Create



Post

```
{  
  "id": 8  
  "title": "Title Seven",  
  "author": "Author One",  
  "category": "science",  
  "rating": 5  
}
```

- Pass in id.
- We want to not include id.
- Have app create id for us.
- All we have to do:
 - Title
 - Author
 - Category
 - Rating

BookRequest Object

```
public class BookRequest {  
    private String title;  
    private String author;  
    private String category;  
    private int rating;  
  
    public BookRequest(String title, String author, String category, <...> ) {  
        this.title = title;  
        this.author = author;  
        this.category = description;  
        <...>  
    }  
  
    < Getters and Setters >  
}
```

- Dedicated Book Object for sending a request to server

POST Request Method

REQUEST:

URL : localhost:8080 / api / books

Create



Post

```
@PostMapping()  
public void createBook(@RequestBody BookRequest bookRequest) {  
    long id;  
    if (books.isEmpty()) {  
        id = 1;  
    } else {  
        id = books.get(books.size() - 1).getId() + 1;  
    }  
  
    Book book = new Book (  
        id, bookRequest.getTitle(), bookRequest.getAuthor,  
        bookRequest.getCategory, bookRequest.getRating()  
    );  
  
    books.add(book);  
}
```


POST Request Method

Create



Post

```
@PostMapping()  
public void createBook(@RequestBody BookRequest bookRequest) {  
    long id;  
    if (books.isEmpty()) {  
        id = 1;  
    } else {  
        id = books.get(books.size() - 1).getId() + 1;  
    }  
    Book book = convertToBook(id, bookRequest);  
  
    books.add(book);  
}
```

```
private Book convertToBook(long id, BookRequest bookRequest) {  
    return new Book(id, bookRequest.getTitle(), bookRequest.getAuthor(),  
                    bookRequest.getCategory(), bookRequest.getRating());  
}
```


POST Request Method

```
@PostMapping
public void createBook(@RequestBody BookRequest bookRequest) {

    long id = books.isEmpty() ? 1 : books.get(books.size() - 1).getId() + 1;

    Book newBook = convertToBook(id, bookRequest);

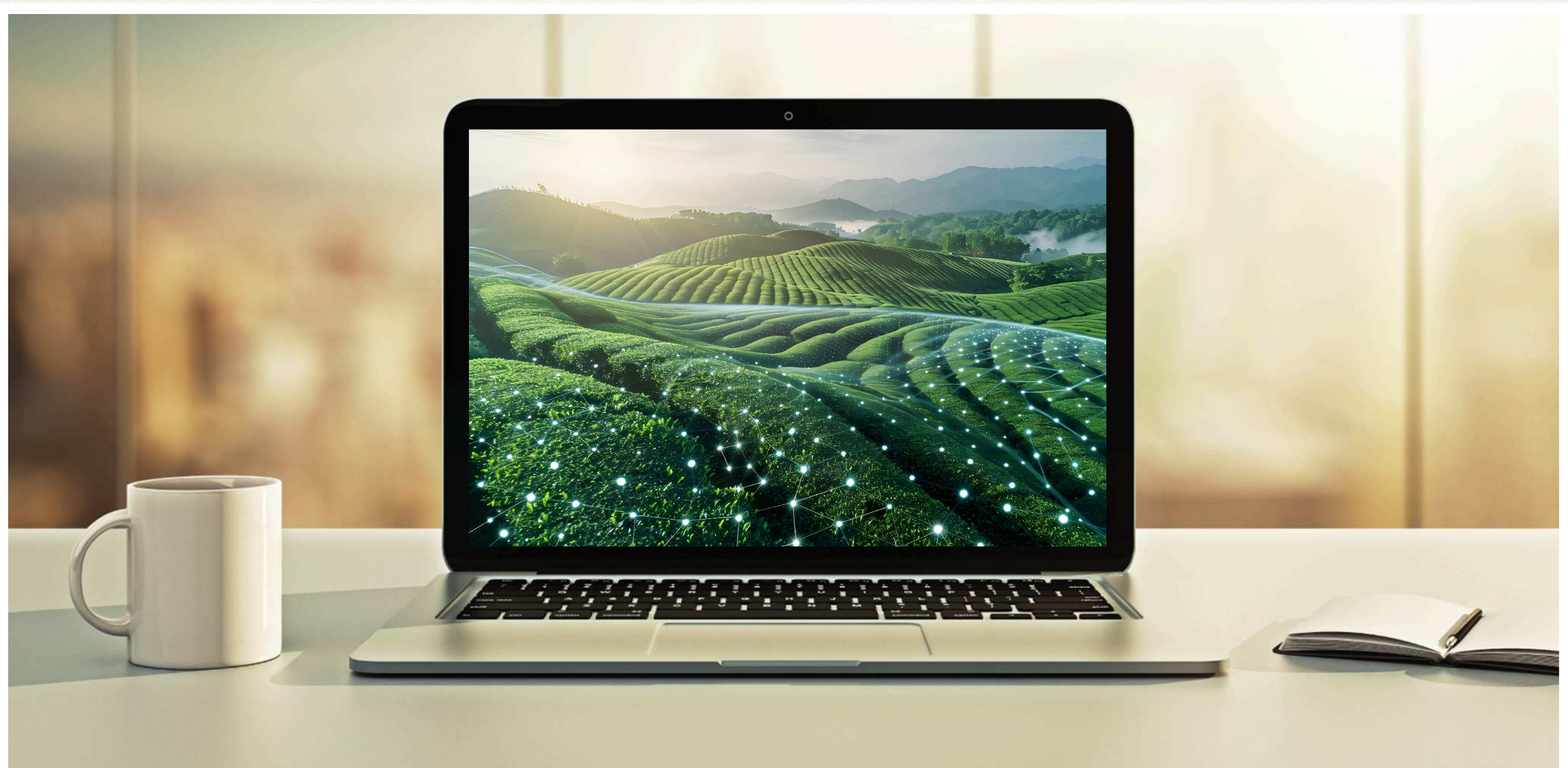
    books.add(newBook);
}
```



```
private Book convertToBook(long id, BookRequest bookRequest) {

    return new Book(id, bookRequest.getTitle(), bookRequest.getAuthor(),
                    bookRequest.getCategory(), bookRequest.getRating());
}
```


Put Request Update



Updates to Put Request

- Consumes Book Entity.
 - We want to change to BookRequest.
- Have it use our convertToBook functionality where we can pass in the:
 - id from the path variable
 - BookRequest object.
- Update current book.

PUT Request Method

Update



Put

REQUEST:

URL : localhost:8080 / api / books / 2

BookRequest

```
@PutMapping("/{id}")
public void updateBook(@PathVariable long id, @RequestBody BookRequest bookRequest) {
    for (int i = 0; i < books.size(); i++) {
        if (book.get(i).getId() == id) {
            Book updatedBook = convertToBook(id, bookRequest);
            books.set(i, updatedBook);
            return;
        }
    }
}
```


Data Validation



What is Data Validation

- Process of ensuring that data is accurate, complete and compliant with rules and formats.
- Check specific data (incoming requests) to validate they are within specific constraints.
- Validation is crucial in maintaining data integrity, preventing errors and ensuring data is correct.

Main Types of Data Validation

- **Presence Validation:** Ensures that required fields are not null or empty.
- **Range Validation:** Ensures that numerical values fall within a specified range.
- **Format Validation:** Checks if data follows appropriate formats (email, phone number, etc.)

Path Variables Validation

Read



Get

- Problem:
- Pass any data at an id
- Entire function runs until no matches are found.

URL : localhost:8080 / api / books / 2

```
@GetMapping("/{id}")
public Book getBookById(@PathVariable long id) {

    return book.stream()
        .filter(book -> book.getId() == id)
        .findFirst()
        .orElse(null);
}
```

0

-10

-1001

Path Variables Validation

- Implement data validation on id

Read



Get

```
@GetMapping("/{id}")
public Book getBookById(@PathVariable long id) {

    if (id < 1) {
        return null;
    }

    return book.stream()
        .filter(book -> book.getId() == id)
        .findFirst()
        .orElse(null);
}
```

Guard Clause

Path Variables Validation

- Use validation package from Jakarta

Read



Get

```
@GetMapping("/{id}")  
public Book getBookById(@PathVariable @Min(value = 1) long id) {  
  
    return book.stream()  
        .filter(book -> book.getId() == id)  
        .findFirst()  
        .orElse(null);  
}
```

Data Validation or throws
an error

Also works for query
parameters

Popular Validations

- `jakarta.validation.constraints`
 - `@NotEmpty`
 - Ensures that the field is not null or empty. It can be applied to `String`, `Collection`, `Map`, or `Array` types. It checks that the field has at least one element or character.
 - `@Size`
 - Defines a constraint on the size of a field. It can be applied to `String`, `Collection`, `Map`, or `Array` types. The annotation allows you to specify the minimum and maximum size (or length) allowed.

Popular Validations

- `jakarta.validation.constraints`
 - `@Min`
 - Ensures that the num value of the field is at least the specified minimum value. It can be applied to numeric types like int, long, float, double, etc.
 - `@Max`
 - Ensures that the numeric value of the field does not exceed the specified maximum value. It can be applied to numeric types like int, long, float, double, etc

Data validation on Request Objects

- Data validation can be added on Request Objects not just Path and Query Parameters.
- BookRequest object has 4 fields: Title, Author, Category, Rating.
- We want to add data validation on the BookRequest to ensure data quality before processing.

How to implement validation?

- jakarta.validation.constraints
 - @Size
 - @Min
 - @Max

```
@Size(min=1, max=30)  
private String title;
```

```
@Size(min=1, max=40)  
private String author;
```

```
@Size(min=1, max=30)  
private String category;
```

```
@Min(value=1)  
@Max(value=5)  
private int rating;
```


Update BookRequest Entity

- Request that comes from client for validation

```
public class BookRequest {  
  
    @Size(min=1, max=30)  
    private String title;  
  
    @Size(min=1, max=40)  
    private String author;  
  
    @Size(min=1, max=30)  
    private String category;  
  
    @Min(value=1)  
    @Max(value=5)  
    private int rating;  
  
    < Constructor, Getters & Setters >  
}
```


BookRequest Validation

- Add @Valid to create the validation on the RequestBody.

```
@PostMapping
public Book createBook(@Valid @RequestBody BookRequest bookRequest) {

    long id = books.isEmpty() ? 1 : books.get(books.size() - 1).getId() + 1;

    Book newBook = convertToBook(id, bookRequest);

    books.add(newBook);
}
```


@ResponseStatus



What is @ResponseStatus

- Used to specify the HTTP status code that should be returned by a handler method or an exception class.
- Typically applied to methods in a controller or directly on an exception class.
- Customizing HTTP Status Codes:
 - Customize the HTTP response status for certain conditions, like returning 404 Not Found when a resource is not found or 201 Created when a new resource is successfully created.

What is @ResponseStatus

```
@ResponseStatus(HttpStatus.NOT_FOUND)
@GetMapping("/resource-not-found")
public void resourceNotFound() {
    // Logic that returns a resource not found
}
```

```
@ResponseStatus(HttpStatus.CREATED)
@PostMapping
public void createBook(@RequestBody BookRequest bookRequest) {

    long id = books.isEmpty() ? 1 : books.get(books.size() - 1).getId() + 1;

    Book newBook = convertToBook(id, bookRequest);

    books.add(newBook);
}
```


Swagger Clean Up



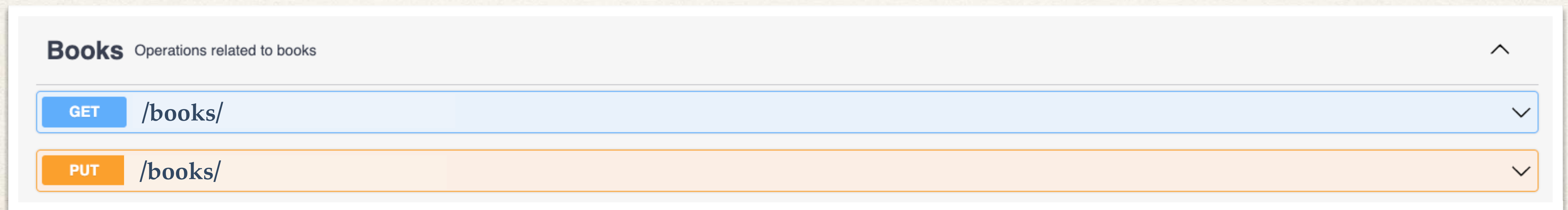
Swagger Clean Up

- Adding more logic to our Swagger UI
 - @Tag
 - Ability to see why the group of endpoints are used for.
 - @Operation
 - Ability to see why an endpoint exists.
 - @Parameter
 - Ability to see why a parameter must be included

Swagger @Tag

- Adding more logic to our Swagger UI
- @Tag
 - Ability to see why the group of endpoints are used for.

```
@Tag(name="Books", description="Operations related to books")
```



Swagger @Operation

- Adding more logic to our Swagger UI
- @Operation
 - Ability to see why an endpoint exists.

```
@Operation(summary="Get all books", description="Retrieve list of all available books")
```

Books Operations related to books					
GET	/books/	Get all books		⌵	
PUT	/books/	Update a book		⌵	

Swagger @Parameter

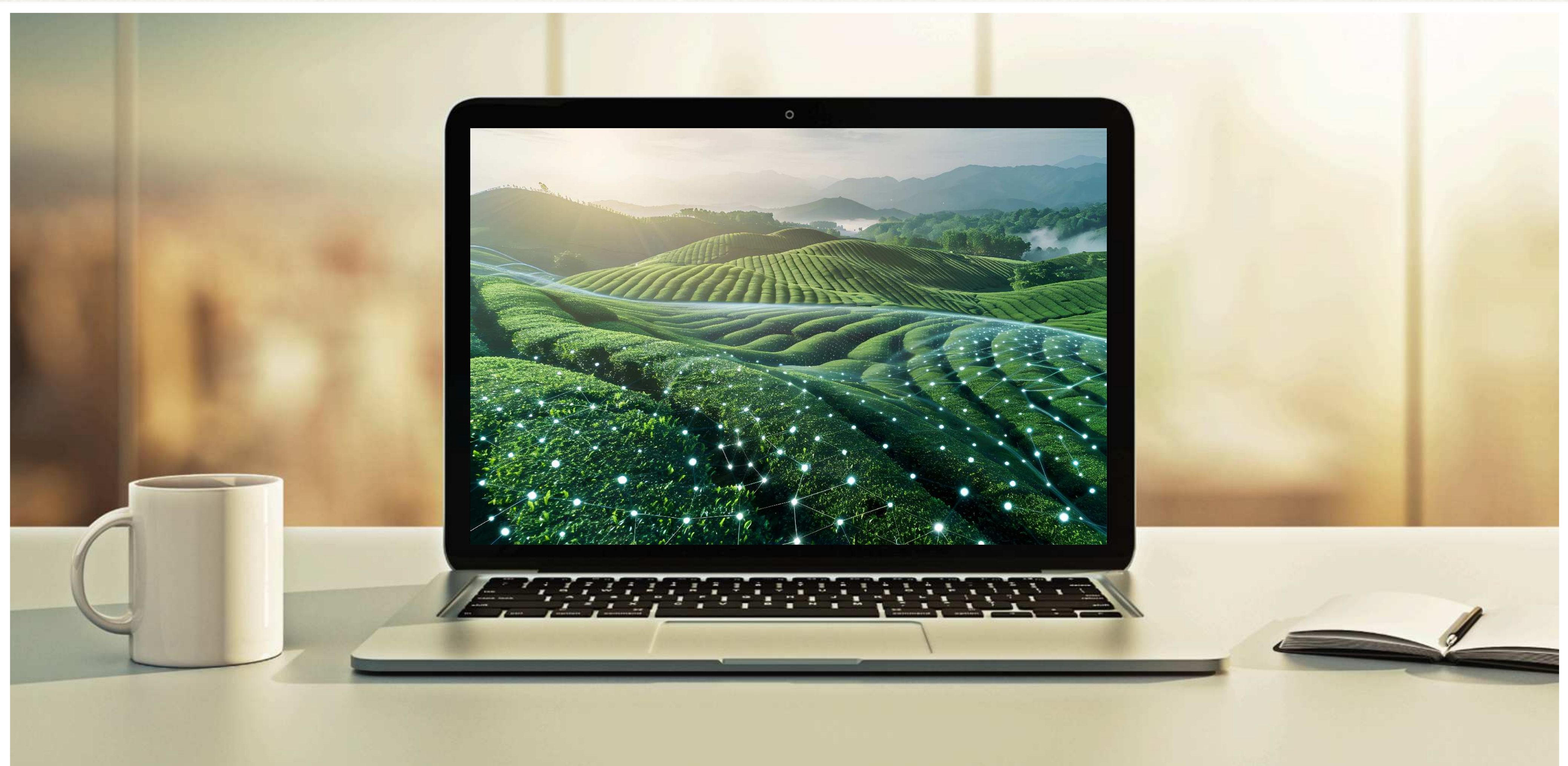
- Adding more logic to our Swagger UI
- @Parameter
 - Ability to see why a parameter must be included

```
@Parameter(description="Title of the book to retrieve")
```

The image shows a snippet of a Swagger UI interface. At the top, it displays a GET method for the endpoint `/books/{title}` with the description "Get book by title". Below this, a section titled "Parameters" is shown. It contains a table with two columns: "Name" and "Description". The first row in the table is for the "title" parameter, which is marked as "required" with a red asterisk. The type is "string" and the location is "(path)". The description is "Title of the book to retrieve". Below the table, there is a text input field with the placeholder text "title".

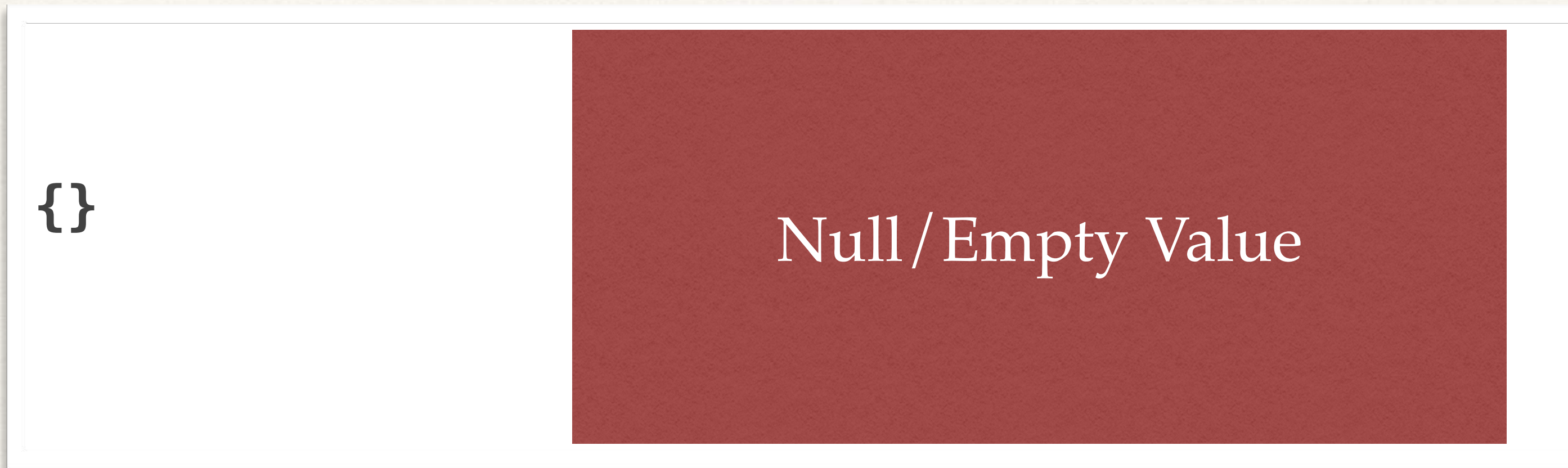
Name	Description
title * required string (path)	Title of the book to retrieve

Spring REST - Exception Handling



What happens with a bad value?

- Bad book id of 9999 ...



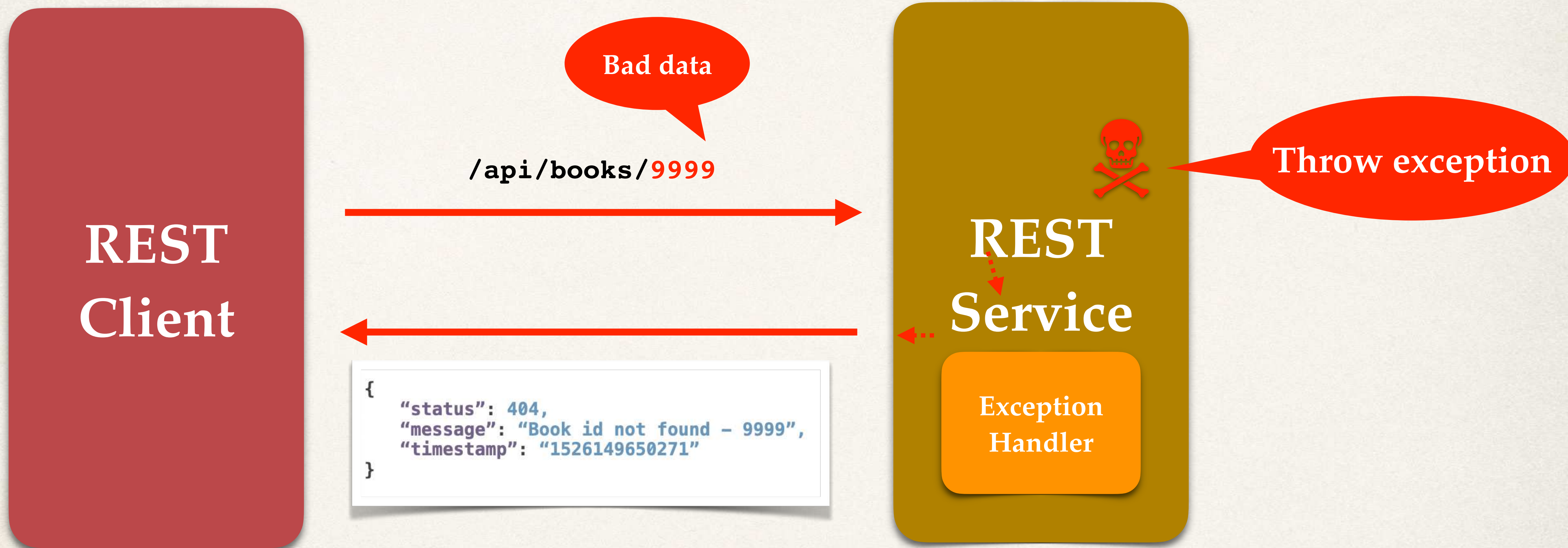
- This is not what we want to happen.
- We want to return something useful by throwing a Book Not Found Exception.

We really want this ...

- Handle the exception and return error as JSON

```
{  
  "status": 404,  
  "message": "Book id not found - 9999",  
  "timestamp": "1526149650271"  
}
```


Spring REST Exception Handling



Development Process

Step-By-Step

1. Create a custom error response class
2. Create a custom exception class
3. Update REST service to throw exception if book not found
4. Add an exception handler method using `@ExceptionHandler`

Step 1: Create custom error response class

- The custom error response class will be sent back to client as JSON

- We will define as Java class (POJO)

BookErrorResponse
status : int message : String timeStamp : long
getStatus() : int setStatus(...) : void ...

You can define any custom fields that you want to track

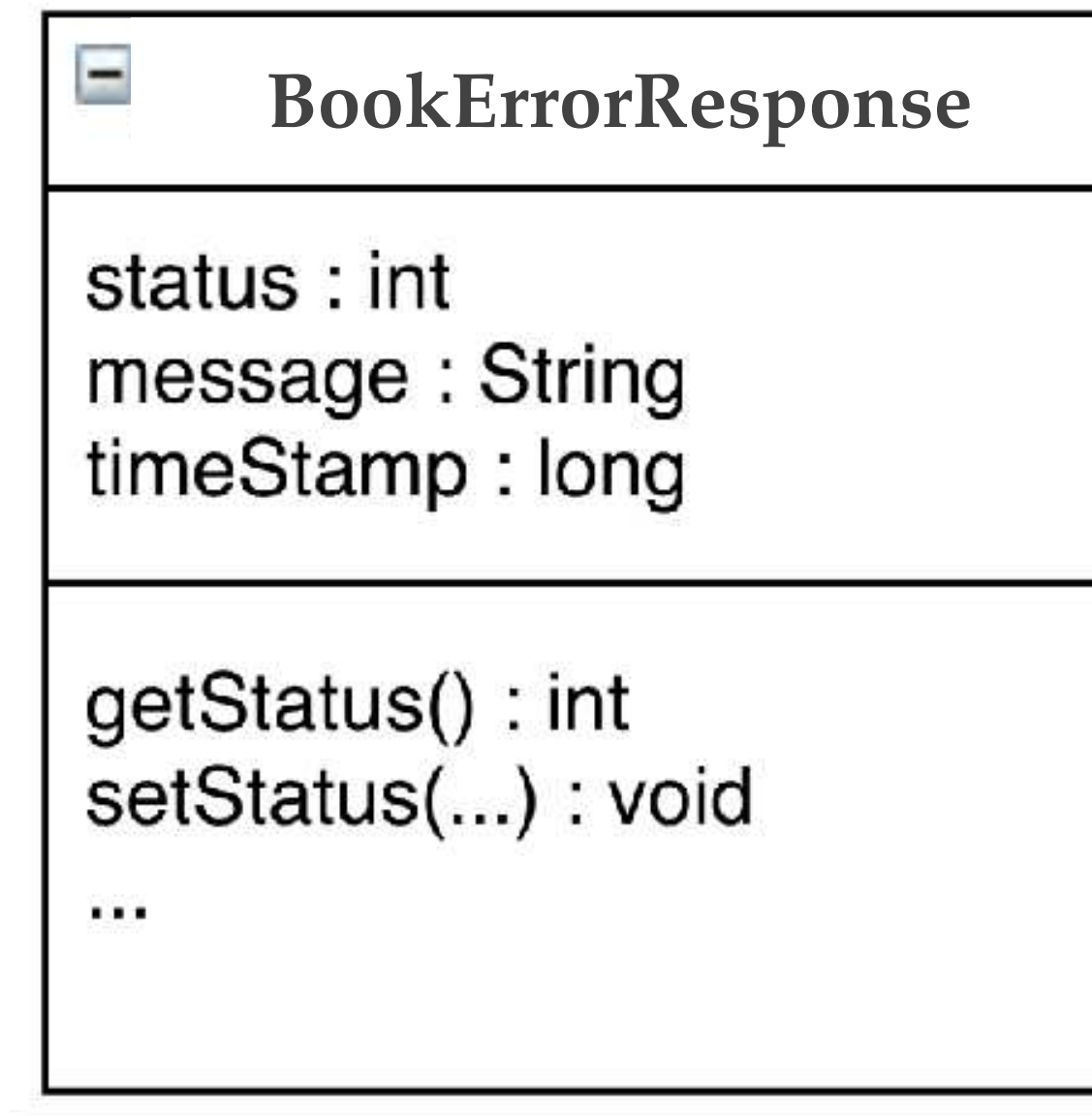
- Jackson will handle converting it to JSON

```
{  
  "status": 404,  
  "message": "Book id not found - 9999",  
  "timestamp": "1526149650271"  
}
```


Step 1: Create custom error response class

File: BookErrorResponse.java

```
public class BookErrorResponse {  
  
    private int status;  
    private String message;  
    private long timeStamp;  
  
    // constructors  
  
    // getters / setters  
  
}
```



```
{  
    "status": 404,  
    "message": "Book id not found - 9999",  
    "timestamp": "1526149650271"  
}
```


Step 2: Create custom book exception

- The custom book exception will be used by our REST service
- In our code, if we can't find book, then we'll throw an exception
- Need to define a custom book exception class
 - `BookNotFoundException`

Step 2: Create custom book exception

File: BookNotFoundException.java

```
public class BookNotFoundException extends RuntimeException {  
  
    public BookNotFoundException(String message) {  
        super(message);  
    }  
  
}
```



Call super class
constructor

Step 3: Update REST service to throw exception

File: BookController.java

```
@RestController
@RequestMapping("/api/books")
public class BookController {

    @GetMapping("/{id}")
    public Book getBookById(@PathVariable long id) {

        // check the id against list size

        return books.stream()
            .filter(book -> book.getId() == id)
            .findFirst()
            .orElseThrow(() -> new BookNotFound("Book id not found - " + bookId));

    }
    ...
}
```

Happy path

Could also
check results
from DB



Throw exception

Spring REST Exception Handling



Step 4: Add exception handler method

- Define exception handler method(s) with `@ExceptionHandler` annotation
- Exception handler will return a `ResponseEntity`
- `ResponseEntity` is a wrapper for the HTTP response object
- `ResponseEntity` provides fine-grained control to specify:
 - HTTP status code, HTTP headers and Response body

Step 4: Add exception handler method

File: Book

```
@RestController
@RequestMapping("/api/books")
public class BookRestController {
    ...

    @ExceptionHandler
    public ResponseEntity<BookErrorResponse> handleException(BookNotFoundException exc) {

        BookErrorResponse error = new BookErrorResponse();

        error.setStatus(HttpStatus.NOT_FOUND.value());
        error.setMessage(exc.getMessage());
        error.setTimeStamp(System.currentTimeMillis());

        return new ResponseEntity<>(error, HttpStatus.NOT_FOUND);
    }
}
```

Exception handler method

Type of the response body

Exception type to handle / catch

Body

Status code

```
{
  "status": 404,
  "message": "Book id not found - 9999",
  "timestamp": "1526149650271"
}
```

BookErrorResponse	
status : int message : String timeStamp : long	
getStatus() : int setStatus(...) : void ...	

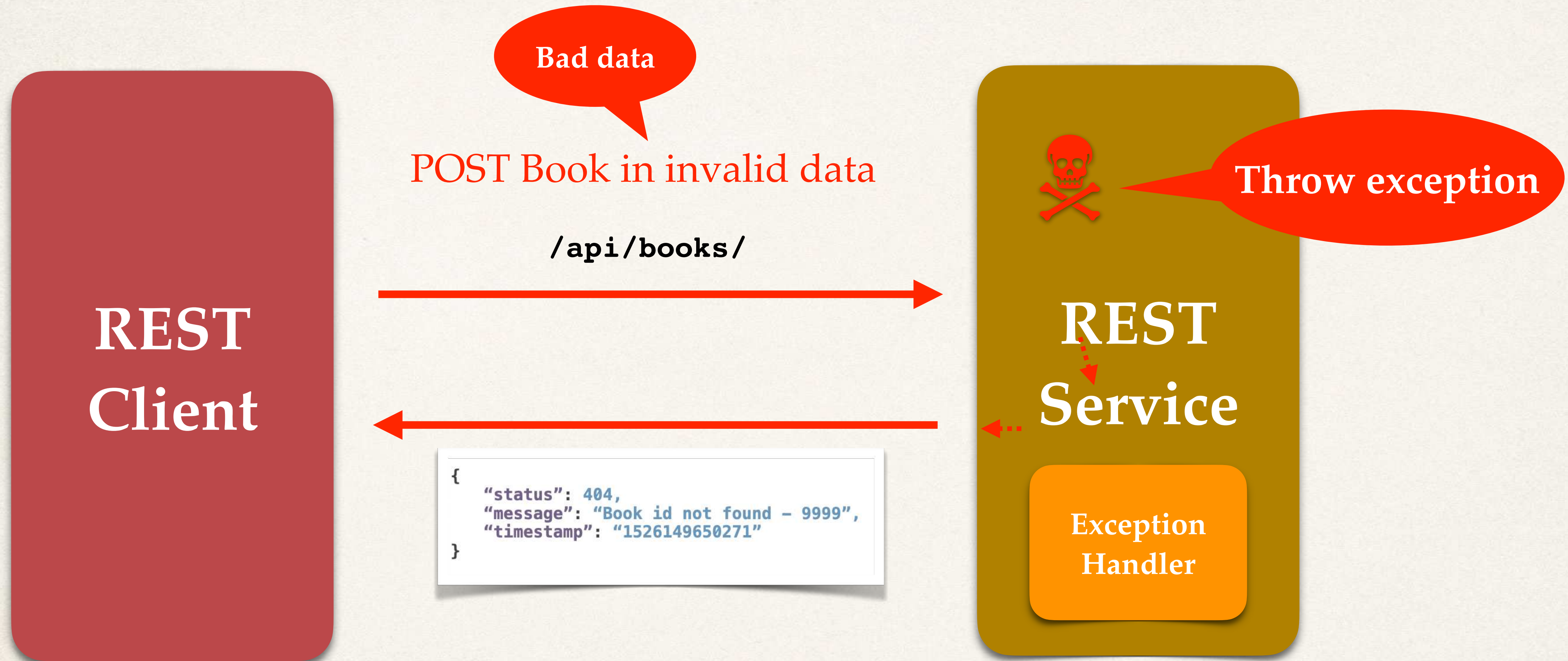
Spring REST Exception Handling



Spring REST - Global Exception Handling

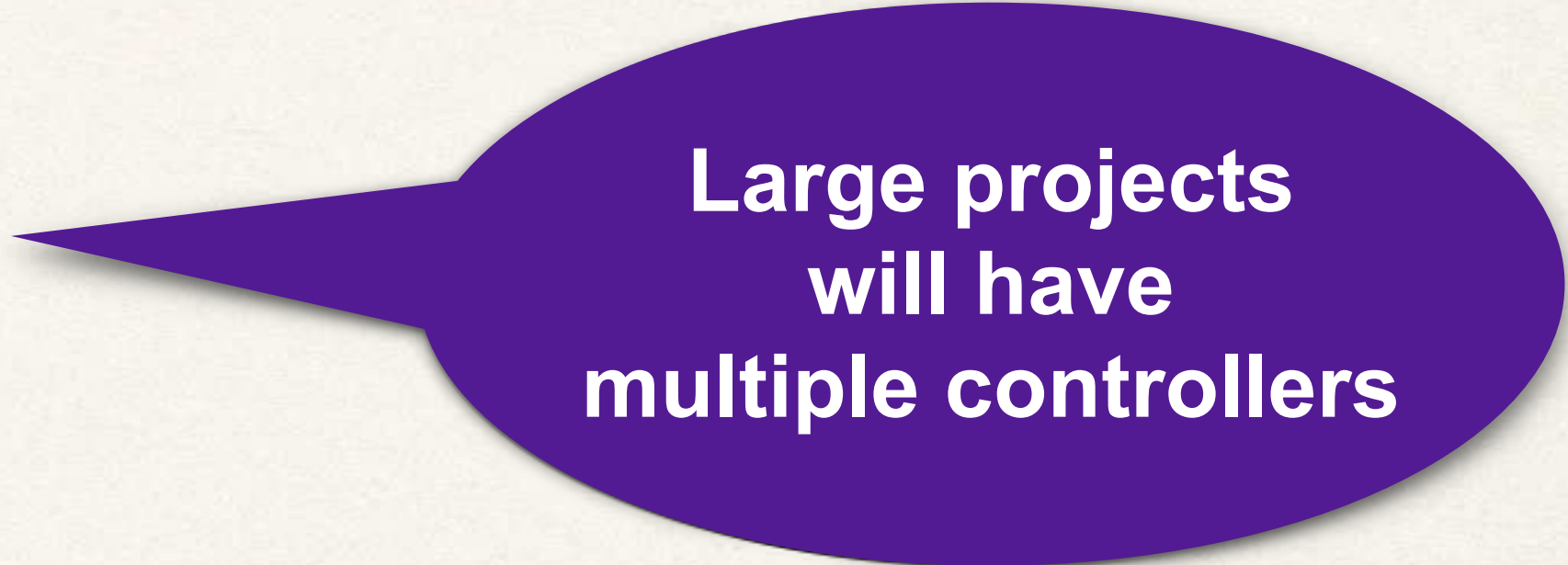


Spring REST Exception Handling



It works, but ...

- Exception handler code is only for the specific REST controller
- Can't be reused by other controllers :-(
- We need **global** exception handlers
 - Promotes reuse
 - Centralizes exception handling



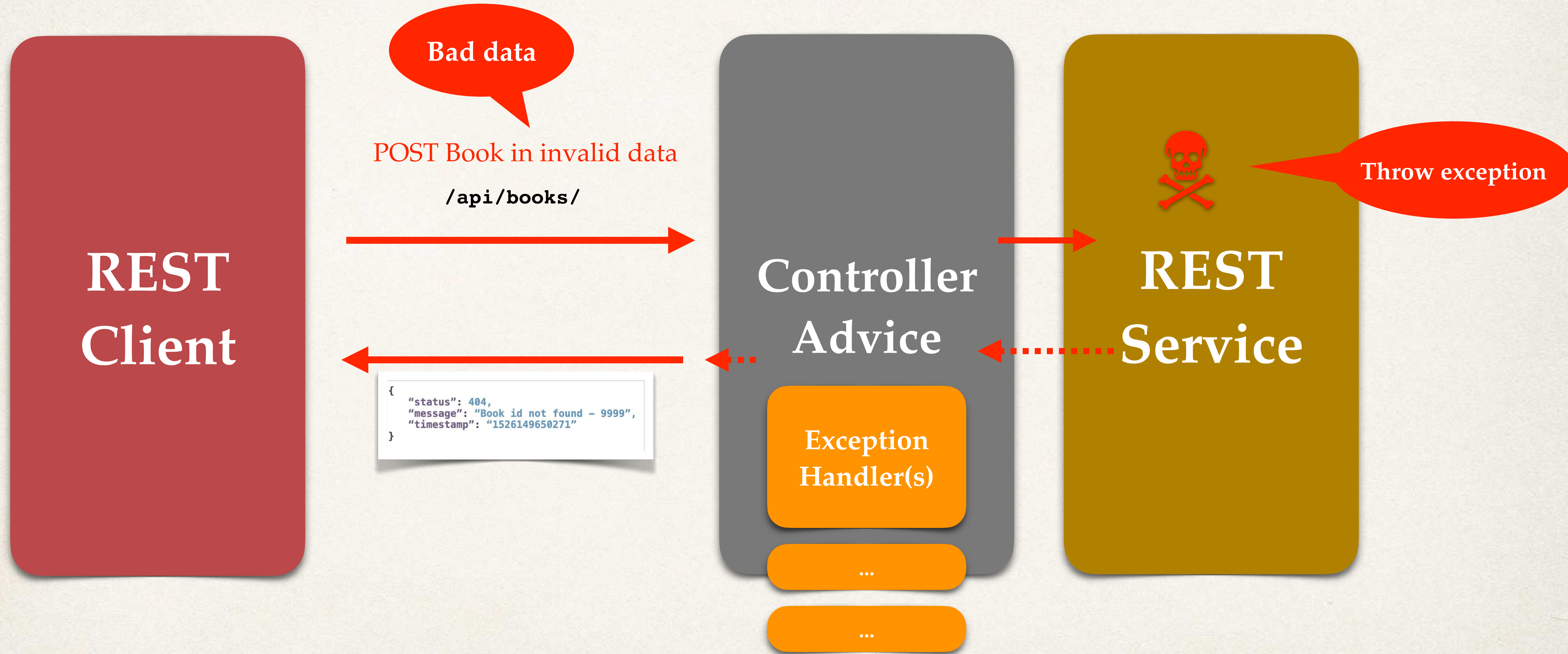
Large projects
will have
multiple controllers

Spring @ControllerAdvice

- @ControllerAdvice is similar to an interceptor / filter
- Pre-process requests to controllers
- Post-process responses to handle exceptions
- Perfect for global exception handling

**Real-time
use of
AOP**

Spring REST Exception Handling



Development Process

Step-By-Step

1. Create new `@ControllerAdvice`
2. Refactor REST service ... remove exception handling code
3. Add exception handling code to `@ControllerAdvice`

Step 1: Create new @ControllerAdvice

File: BookRestExceptionHandler.java

```
@ControllerAdvice
public class BookRestExceptionHandler {

    ...

}
```


Step 2: Refactor - remove exception handling

File: BookRestController.java

```
@RestController
@RequestMapping("/api/books")
public class BookRestController {
    ...

    @ExceptionHandler
    public ResponseEntity<BookErrorResponse> handleException(BookNotFoundException exc) {

        BookErrorResponse error = new BookErrorResponse();

        error.setStatus(HttpStatus.NOT_FOUND.value());
        error.setMessage(exc.getMessage());
        error.setTimestamp(System.currentTimeMillis());

        return new ResponseEntity<>(error, HttpStatus.NOT_FOUND);
    }
}
```



Remove
this code

Step 3: Add exception handler to @ControllerAdvice

File: BookRestExceptionHandler.java

Same code
as before

```
@ControllerAdvice
public class BookRestExceptionHandler {

    @ExceptionHandler
    public ResponseEntity<BookErrorResponse> handleException(BookNotFoundException exc) {

        BookErrorResponse error = new BookErrorResponse();

        error.setStatus(HttpStatus.NOT_FOUND.value());
        error.setMessage(exc.getMessage());
        error.setTimestamp(System.currentTimeMillis());

        return new ResponseEntity<>(error, HttpStatus.NOT_FOUND);
    }
}
```


Spring REST Exception Handling

